

RAG Interview Q&A: How Retrieval-Augmented Generation Actually Works

A practical guide to RAG, Advanced RAG, Graph RAG, Hybrid RAG, Multimodal RAG, and Agentic RAG



AI ENGINEERING INSIDER

MAY 18, 2026



Retrieval-Augmented Generation, or **RAG**, is one of the most important architectures in modern AI engineering.

It powers enterprise search, internal knowledge assistants, customer support bots, documentation copilots, legal research tools, healthcare assistants, and many other production AI systems.

This Substack is reader-supported. To

receive new posts and support my work,
consider becoming a free or paid subscriber.

✓ Subscribed

But what exactly is RAG? Why do we need it? How does it work? And how do different RAG architectures compare?

Q1. What is RAG?

RAG stands for Retrieval-Augmented Generation.

In simple terms:

- ↳ **Retrieval** means finding the most relevant documents, chunks, or information based on a user's question.
- ↳ **Generation** means using an LLM to generate an answer using that retrieved information.

So instead of asking the LLM to answer only from its internal training data, RAG gives the model fresh and relevant context before it responds.

A simple way to remember it:

RAG = Retrieval + Generation

Q2. Why do we need RAG?

LLMs like GPT-4, Claude, Gemini, DeepSeek, and other foundation models are powerful, but they have limitations.

Most LLMs are trained on broad public data such as:

- ↳ Wikipedia
- ↳ Books
- ↳ Public websites
- ↳ Common Crawl data
- ↳ General internet-scale datasets

The problem is that enterprise knowledge is usually private, dynamic, and domain-specific.

For example:

- ↳ Internal company policies
- ↳ Technical documentation
- ↳ Product manuals
- ↳ Legal contracts
- ↳ Financial reports
- ↳ Customer records
- ↳ Medical guidelines
- ↳ Engineering design documents

A standard LLM may not know this information. Even worse, when it does not know the answer, it may hallucinate.

That means it can generate an answer that sounds confident but is actually incorrect.

RAG helps solve this problem by grounding the LLM's response in retrieved documents.

Q3. How does RAG reduce hallucination?

RAG reduces hallucination by giving the LLM relevant supporting context at runtime.

Instead of relying only on what the model “remembers,” the system first searches a knowledge base and retrieves the most relevant information.

Then the LLM uses that retrieved context to generate an answer.

This improves reliability because the answer is grounded in actual documents.

A good RAG system can also include citations, so users can verify where the answer came from.

That is why RAG is useful in enterprise AI systems where accuracy, trust, and traceability matter.

Q4. How does a basic RAG system work?

A RAG system usually has two major stages:

- ↳ **Data Indexing**

- ↳ **Retrieval + Generation**

The first stage prepares the data.

The second stage uses that data to answer user questions.

Q5. What happens during data indexing?

Data indexing is the preparation step.

Here is the process:

- ↳ First, collect the source documents. These could be PDFs, Word files, PowerPoint slides, Excel sheets, web pages, or internal documentation.

- ↳ Next, split the documents into smaller pieces called **chunks**.
- ↳ Then, pass those chunks through an **embedding model**.
- ↳ The embedding model converts each chunk into a vector representation.
- ↳ Finally, store those vectors inside a vector database such as Milvus, Pinecone, Weaviate, Chroma, Qdrant, or pgvector.

The goal is to make the documents searchable by meaning, not just by exact keywords.

Q6. What is an embedding in RAG?

An embedding is a numerical representation of text.

It converts words, sentences, paragraphs, or document chunks into vectors.

These vectors capture semantic meaning.

For example, these two questions may have different wording:

- ↳ “How do I reset my password?”
- ↳ “I forgot my login credentials. What should I do?”

A keyword search might treat them differently.

But an embedding model can understand that both questions are semantically related.

That is why embeddings are important in RAG.

Q7. What happens during retrieval and generation?

When a user asks a question, the RAG system follows this flow:

- ↳ The user submits a question.
- ↳ The question is passed through the same embedding model.
- ↳ The question is converted into a vector.
- ↳ The system compares that vector with stored document vectors in the vector database.
- ↳ The most similar chunks are retrieved.
- ↳ These chunks are added to the prompt as context.
- ↳ The LLM generates an answer based on the question and retrieved context.

This is the core RAG pipeline.

Q8. What is top-k retrieval?

Top-k retrieval means retrieving the top number of most relevant chunks from the vector database.

For example:

- ↳ Top-3 retrieval returns the 3 most relevant chunks.
- ↳ Top-5 retrieval returns the 5 most relevant chunks.
- ↳ Top-10 retrieval returns the 10 most relevant chunks.

Choosing the right value of **k** matters.

If **k** is too small, the system may miss important context.

If **k** is too large, the prompt may become noisy, expensive, and less focused.

A strong RAG system balances relevance, cost, latency, and context quality.

Q9. What is Naïve RAG?

Naïve RAG is the simplest RAG architecture.

It works like this:

- ↳ User asks a question.
- ↳ The system retrieves relevant chunks from a vector database.
- ↳ The retrieved chunks are passed to the LLM.
- ↳ The LLM generates an answer.

This architecture is easy to build and useful for many simple use cases.

However, it has limitations.

Naïve RAG may retrieve irrelevant chunks, miss important information, struggle with complex queries, and produce weak answers when the retrieved context is poor.

Q10. What is Advanced RAG?

Advanced RAG improves the basic RAG pipeline by optimizing what happens before and after retrieval.

It usually includes:

- ↳ Pre-retrieval strategies
- ↳ Post-retrieval strategies
- ↳ Better chunking
- ↳ Re-ranking

- ↳ Query rewriting
- ↳ Query expansion
- ↳ Context compression
- ↳ Fine-tuned embedding models

The goal is to improve the quality of the retrieved context before it reaches the LLM.

Better retrieval usually leads to better generation.

Q11. What is query rewriting?

Query rewriting improves the user's original question before retrieval happens.

For example, users often ask unclear, messy, or incomplete questions.

Query rewriting can help by:

- ↳ Fixing grammar
- ↳ Correcting typos
- ↳ Simplifying complex phrasing
- ↳ Rephrasing the question for clarity
- ↳ Making the query more search-friendly

Example:

Original question:

"Why my payment no work yesterday?"

Rewritten query:

"Why did my payment fail yesterday?"

This helps the retriever find better matches.

Q12. What is query expansion?

Query expansion creates multiple variations of the original query.

This helps the system search more broadly.

For example, if the user asks:

“How do I cancel my subscription?”

The system might expand it into:

- ↳ “Cancel subscription”
- ↳ “Stop recurring billing”
- ↳ “End membership”
- ↳ “Disable auto-renewal”
- ↳ “Close account plan”

This improves the chance of retrieving relevant documents, especially when different documents use different terminology.

Q13. What is re-ranking in RAG?

Re-ranking happens after retrieval.

The vector database may return a set of candidate chunks, but not all of them are equally useful.

A re-ranker reviews those chunks and reorders them based on relevance to the user’s question.

The most useful chunks are placed higher in the prompt.

This matters because LLMs are sensitive to context ordering.

If the most relevant information is buried under weaker content, the final answer may suffer.

Q14. What is context compression?

Context compression reduces the size of retrieved content before sending it to the LLM.

It removes:

- ↳ Redundant content
- ↳ Off-topic text
- ↳ Low-value details
- ↳ Repeated information
- ↳ Unnecessary filler

This helps the system fit more useful information into the model's context window.

It also reduces cost and improves answer focus.

Q15. When should you fine-tune an embedding model?

Fine-tuning an embedding model can be useful when working with specialized domains.

Examples include:

- ↳ Medical documents
- ↳ Legal contracts
- ↳ Scientific papers
- ↳ Cybersecurity reports
- ↳ Financial regulations
- ↳ Internal enterprise terminology

General embedding models may not understand niche language well.

A fine-tuned embedding model can improve semantic matching for domain-specific vocabulary, abbreviations, and concepts.

However, fine-tuning should not be the first step.

Before fine-tuning, it is usually better to improve chunking, metadata, query rewriting, retrieval strategy, and evaluation.

Q16. What is Graph RAG?

Graph RAG combines RAG with knowledge graphs.

Traditional RAG retrieves chunks based on vector similarity.

Graph RAG adds relationships between entities.

For example, a knowledge graph can represent:

- ↳ People
- ↳ Products
- ↳ Organizations
- ↳ Policies
- ↳ Events
- ↳ Dependencies
- ↳ Relationships between entities

This allows the system to retrieve information based not only on semantic similarity, but also on relationships.

Graph RAG is useful when the answer depends on connected information.

Q17. When is Graph RAG useful?

Graph RAG is useful when relationships matter.

For example:

- ↳ “Which supplier is connected to this delayed shipment?”
- ↳ “Which policy affects this compliance requirement?”
- ↳ “Which customer accounts are impacted by this product issue?”
- ↳ “Which internal systems depend on this API?”
- ↳ “Which entities are connected to this financial transaction?”

Vector search may find similar text.

Graph RAG can reason over connected entities.

That makes it powerful for complex enterprise knowledge systems.

Q18. What is Hybrid RAG?

Hybrid RAG combines multiple retrieval methods.

A common version combines:

- ↳ Vector search
- ↳ Keyword search

Vector search is good at semantic meaning.

Keyword search is good at exact matches.

For example, keyword search is useful for:

- ↳ Product IDs
- ↳ Error codes
- ↳ Legal clause numbers
- ↳ Customer IDs
- ↳ API names
- ↳ Exact technical terms

Hybrid RAG gives you the benefit of both semantic retrieval and exact-match retrieval.

Q19. What is the difference between Naïve RAG and Hybrid RAG?

Naïve RAG usually depends mainly on vector search.

Hybrid RAG combines vector search with another method, such as keyword search or graph retrieval.

Naïve RAG is simpler.

Hybrid RAG is usually more reliable for enterprise use cases because real-world queries often contain both semantic intent and exact-match requirements.

For example:

“Explain error code AUTH-403 in the payment API.”

A vector-only system may retrieve general authentication content.

A hybrid system can retrieve documents that specifically contain **AUTH-403**.

Q20. What is Multimodal RAG?

Multimodal RAG retrieves and uses more than just text.

It can work with:

- ↳ Text
- ↳ Images
- ↳ Tables
- ↳ Charts
- ↳ Screenshots
- ↳ Audio transcripts
- ↳ Video frames
- ↳ Diagrams

This is useful when important information is stored in visual or mixed formats.

For example, a product manual may include diagrams.

A financial report may include tables and charts.

A medical document may include scanned images.

Multimodal RAG helps the system answer using richer context.

Q21. What is Agentic RAG?

Agentic RAG uses agents to control retrieval and reasoning.

Instead of a fixed pipeline, an agent can decide:

- ↳ Which data source to search
- ↳ Which tool to call
- ↳ Whether more retrieval is needed
- ↳ Whether the answer is grounded
- ↳ Whether to ask a follow-up question
- ↳ Whether to escalate to a human

This makes Agentic RAG more dynamic than traditional RAG.

It is especially useful when the system needs to interact with multiple tools, APIs, databases, or knowledge sources.

Q22. What is Single-Agent RAG?

Single-Agent RAG uses one agent to manage the retrieval process.

The agent may act as a router.

It decides which source to query, such as:

- ↳ Vector database
- ↳ SQL database
- ↳ API
- ↳ Web search
- ↳ Internal documentation
- ↳ CRM system

This is useful when one intelligent controller can manage the retrieval workflow.

Q23. What is Multi-Agent RAG?

Multi-Agent RAG uses multiple specialized agents.

For example:

- ↳ A document retrieval agent
- ↳ A database agent
- ↳ A web search agent
- ↳ A compliance agent
- ↳ A summarization agent
- ↳ A verification agent

A master agent or orchestrator coordinates these agents.

This architecture is useful when the task is complex and requires multiple specialized workflows.

Q24. What is the biggest mistake people make when building RAG?

The biggest mistake is thinking RAG is only about connecting a vector database to an LLM.

Production RAG is much more than that.

A strong RAG system needs:

- ↳ Good document ingestion
- ↳ Clean chunking strategy
- ↳ Metadata filtering
- ↳ Hybrid retrieval
- ↳ Re-ranking
- ↳ Evaluation

- ↳ Observability
- ↳ Guardrails
- ↳ Citation support
- ↳ Latency optimization
- ↳ Cost control
- ↳ Feedback loops

RAG is not just a retrieval pipeline.

It is an information quality system.

Q25. How should engineers evaluate a RAG system?

A RAG system should be evaluated at multiple levels.

Key evaluation areas include:

- ↳ **Retrieval quality:** Did the system retrieve the right documents?
- ↳ **Answer faithfulness:** Is the answer grounded in the retrieved context?
- ↳ **Answer relevance:** Does the response answer the user's question?
- ↳ **Citation accuracy:** Do citations support the claims?
- ↳ **Latency:** Is the system fast enough for users?
- ↳ **Cost:** Is the system affordable at scale?
- ↳ **Robustness:** Does it handle ambiguous, messy, or adversarial questions?

Without evaluation, it is difficult to know whether a RAG system is actually working.

Final Takeaway

RAG is one of the core architectures behind modern AI applications.

At a basic level, it retrieves relevant information and gives it to an LLM.

But in production, RAG can become much more advanced.

You may need:

- ↳ Naïve RAG for simple document Q&A
- ↳ Advanced RAG for better retrieval quality
- ↳ Graph RAG for relationship-aware reasoning
- ↳ Hybrid RAG for semantic + keyword retrieval
- ↳ Multimodal RAG for text, images, tables, and diagrams
- ↳ Agentic RAG for dynamic tool use and multi-step workflows

The better your retrieval system, the better your AI application becomes.

Because in RAG, the final answer is only as good as the context you retrieve.

Thanks for reading.

Subscribe for more practical AI engineering breakdowns, system design notes, and interview preparation guides.

This Substack is reader-supported. To receive new posts and support my work, consider becoming a free or paid subscriber.

✓ Subscribed